

Contents

guide_01_memgpt: MemGPT: Towards LLMs as Operating Systems	3
0. 一句话先定住	3
1. 为什么这篇论文在当时重要	3
1.1 2023 的真实痛点	3
1.2 只扩大 context 为什么不够	4
1.3 OS 类比的意义	4
2. 论文到底解决什么问题	4
2.1 长期对话 agent	4
2.2 长文档和多文档分析	5
3. 先掌握 8 个概念	5
3.1 Main context	5
3.2 System instructions	6
3.3 Working context	6
3.4 FIFO queue	6
3.5 Recall storage	7
3.6 Archival storage	7
3.7 Function executor	7
3.8 Heartbeat and yield	7
4. 系统结构重建	7
4.1 状态变量	7
4.2 Context budget	8
4.3 Queue manager 的控制规则	8
4.4 事件驱动	8
4.5 Function call 控制流	9
5. 张量和 token 级别怎么理解	9
5.1 LLM 能看到什么	9
5.2 Attention 视角下的 main memory	9
5.3 Working context 和 recall 的信息粒度	10
6. 方法细节逐段拆	10
6.1 Main context 的三段	10
6.2 External context 的两类	10
6.3 Memory pressure warning	11
6.4 Recursive summary	11
6.5 Pagination	11
6.6 Function errors 也是反馈	11
7. 和 ReAct, Toolformer, MCP 的关系	12
7.1 ReAct	12
7.2 Toolformer	12
7.3 MCP-style tool interface	12
8. 实验总览	12
9. 实验一: Deep Memory Retrieval	13
9.1 任务是什么	13
9.2 Baseline 怎么设	13
9.3 怎么评分	13
9.4 结果数字	13
9.5 这个结果证明了什么	14
9.6 这个结果没有证明什么	14
10. 实验二: Conversation opener	14

10.1 任务是什么	14
10.2 指标是什么	14
10.3 结果数字	14
10.4 怎么读这个结果	14
11. 实验三: Multi-document QA	15
11.1 任务设置	15
11.2 Baseline 和 MemGPT 差别	15
11.3 结果怎么解读	15
11.4 重要限制	16
12. 实验四: Nested key-value retrieval	16
12.1 为什么要做这个任务	16
12.2 实验设置	16
12.3 结果怎么读	16
13. 论文贡献重新归纳	16
13.1 贡献一: Virtual context management	16
13.2 贡献二: OS-inspired memory hierarchy	17
13.3 贡献三: Event-based control flow	17
13.4 贡献四: Function chaining	17
13.5 贡献五: 两类实验验证	17
14. 设计动机和技术权衡	17
14.1 为什么让 LLM 自己管 memory	17
14.2 为什么不是单纯 summary	17
14.3 为什么不是单纯 vector DB	18
14.4 为什么不是直接无限 context	18
15. 本仓库代码对应关系	18
15.1 letta_mock.py	18
15.2 memgpt_virtual_context.py	19
15.3 context_mgmt.py	19
15.4 capstone_memory_chat.py	19
15.5 vector_store.py, episodic_memory.py, semantic_memory.py	20
16. 本地运行路径	20
17. 新手最容易误读的点	20
17.1 “MemGPT 等于 RAG”	20
17.2 “MemGPT 拥有无限上下文”	21
17.3 “只要有 summary 就够”	21
17.4 “工具调用模型越强, MemGPT 就一定越好”	21
17.5 “写入 memory 永远是好事”	21
18. 局限性	21
18.1 依赖 function calling 能力	21
18.2 检索仍然是瓶颈	21
18.3 评测依赖 LLM judge	22
18.4 Memory 写入和更新没有形式化保证	22
18.5 安全和隐私只是开始	22
19. 对现在的意义	22
19.1 为什么 2026 仍然要学	22
19.2 它给 agent 工程的启发	22
19.3 它和本仓库学习路线的关系	23
20. 建议你怎么用 AI agent 学这篇	23
20.1 不要让 agent 直接总结	23
20.2 让 agent 考你状态转移	23

20.3 让 agent 考你实验	24
20.4 让 agent 逼你连代码	24
21. 30 分钟学习任务	24
21.1 第 1 遍: 只画系统图	24
21.2 第 2 遍: 跑 toy code	24
21.3 第 3 遍: 背实验数字	25
21.4 第 4 遍: 改一个变量	25
21.5 第 5 遍: 闭卷复述	25
22. 一页纸复习版	25
22.1 最小机制	25
22.2 最小证据	26
22.3 最小警惕	26
23. 闭卷自测	26
24. 你应该带走的直觉	26

guide_01_memgpt: MemGPT: Towards LLMs as Operating Systems

原论文: MemGPT: Towards LLMs as Operating Systems

本地原文 PDF: learning/agent-memory-context/paper/01_memgpt.pdf

作者: Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, Joseph E. Gonzalez

年份: 2023, PDF metadata 2024-02

本导读目标: 不逐字复制论文, 而是把论文的背景、系统结构、控制流、实验设计、证据链和本仓库代码完整重建出来。读完这篇 guide, 你应该能离开原文复述 MemGPT 的技术主张, 并在本地跑一个玩具版 memory agent。

0. 一句话先定住

MemGPT 的核心不是 “给 LLM 接一个向量库”。

它真正提出的是一个 OS-inspired 的 agent 运行时:

- 把 LLM 的 context window 当成有限 main memory。
- 把对话历史和文档库放到 external context。
- 让 LLM 通过 function calls 主动管理 memory read, write, search, replace。
- 用 memory pressure warning, queue eviction, recursive summary, pagination, heartbeat chaining 等机制, 让固定 context 的模型表现得像有更大的 virtual context。

所以这篇论文的关键词是 virtual context management, 不是单纯 RAG, 也不是单纯 long-context 模型。

1. 为什么这篇论文在当时重要

1.1 2023 的真实痛点

2023 年的 LLM 已经可以做聊天、摘要、问答、代码和工具调用, 但上下文窗口仍然是很硬的边界。论文 Table 1 用 2024-01 收集的数据说明当时常见模型的大致 context 上限:

- Llama 1: 2k tokens, 约 20 条平均消息。

- Llama 2: 4k tokens, 约 60 条平均消息。
- GPT-3.5 Turbo release: 4k tokens, 约 60 条平均消息。
- Mistral 7B: 8k tokens, 约 140 条平均消息。
- GPT-4 release: 8k tokens, 约 140 条平均消息。
- GPT-3.5 Turbo: 16k tokens, 约 300 条平均消息。
- GPT-4: 32k tokens, 约 600 条平均消息。
- Claude 2: 100k tokens, 约 2000 条平均消息。
- GPT-4 Turbo: 128k tokens, 约 2600 条平均消息。
- Yi-34B-200k: 200k tokens, 约 4000 条平均消息。

注意: 这些数字是论文当时的历史上下文, 不是 2026 年的模型规格说明。它们在导读里只承担一个作用: 帮你理解作者为什么认为“直接扩大 context”不是完整答案。

1.2 只扩大 context 为什么不够

新手常见误解是: 既然 context 不够, 那把 context 从 8k 扩到 128k, 1M, 10M 就好了。

论文的判断更细:

- context 增大通常带来更高计算成本和更高延迟。
- 长文档可以轻松超过几十万甚至上百万 token, 例如法律和财务文档。
- 多文档分析要求跨多个文件取证, 不只是把一个文件塞进去。
- 长 context 模型也可能在中间位置的信息利用上变差, 这和当时的 lost-in-the-middle 讨论有关。
- 聊天 agent 的长期记忆不只是“看得见历史”, 还包括何时记、记什么、何时改、何时忘、如何查。

MemGPT 的价值就在这里: 它把问题从“模型最多能看多少 token”转成“系统如何管理有限可见 token 和外部存储之间的数据流”。

1.3 OS 类比的意義

操作系统里, 程序看到的 virtual memory 可以比物理 RAM 大。OS 通过 paging 把暂时不用的数据放到 disk, 需要时再调回 main memory。

MemGPT 借用这个类比:

- LLM context window 类似 RAM。
- external context 类似 disk。
- function calls 类似系统调用。
- memory pressure warning 类似内存压力中断。
- recall 或 archival search 类似从外部存储把页调回主存。
- recursive summary 类似压缩过的旧页面摘要。

这个类比不是为了好听。它直接决定了论文的系统设计: LLM 不只是回答问题, 它还被提示成一个会管理自己记忆的进程。

2. 论文到底解决什么问题

论文聚焦两个固定 context LLM 明显吃亏的场景。

2.1 长期对话 agent

长期对话 agent 需要记住:

- 用户是谁。

- 用户偏好什么。
- 以前发生过什么事件。
- 角色和 persona 是否一致。
- 当前问题是否引用了很久以前的细节。

如果只保留最近 N 条消息，早期事实会消失。

如果只做递归摘要，摘要可能丢掉精确细节，尤其是用户突然问一个很窄的问题时。

如果把全部历史塞进 prompt，成本和 context 都会爆。

MemGPT 的目标是让 agent 能在当前主上下文之外保存过去，并在需要时查回来。

2.2 长文档和多文档分析

长文档问答和多文档分析需要：

- 从很多文档中找相关证据。
- 可能连续翻页。
- 可能根据第一次结果生成第二次查询。
- 可能做 multi-hop lookup。

传统 retriever-reader baseline 通常是：

query

- > retriever returns top K documents
- > concatenate or truncate documents into context
- > LLM answers

这会被 context window 限制。K 太小可能没有金证据，K 太大又塞不进去，截断还会丢答案片段。

MemGPT 的替代路径是：

query

- > all documents live in archival storage
- > LLM calls archival_storage.search
- > result page enters main context
- > LLM decides whether to search again or answer

差别很关键：检索不再是推理前的一次性外部步骤，而是 agent 运行时的一部分。

3. 先掌握 8 个概念

3.1 Main context

Main context 就是本次 LLM inference 真正看得见的 prompt tokens。论文把它拆成三段：

- System instructions.
- Working context.
- FIFO queue.

可以把一次推理的输入想成：

```
main_context_t =
  system_instructions
+ working_context_t
+ fifo_queue_t
```

其中 t 是当前事件或当前 turn。

3.2 System instructions

System instructions 是只读的。它告诉模型:

- MemGPT 的 memory hierarchy 是什么。
- 哪些函数可以调用。
- 每个函数应该在什么场景使用。
- 遇到 memory pressure 时应该保存什么。
- 如何查询 out-of-context data。

在普通聊天模型里, system prompt 只是行为说明。到了 MemGPT, 它变成 agent 的“操作系统说明书”。

3.3 Working context

Working context 是一个固定大小、可读写的文本块, 只能通过 MemGPT functions 修改。

对话场景里, 它通常保存:

- 关于用户的稳定事实。
- 用户偏好。
- agent persona。
- 当前任务目标。
- 需要长期保持的关键状态。

本仓库 `letta_mock.py` 里对应的是 `core`:

```
core = {  
    "human": "User name is Alice",  
    "persona": "helpful agent",  
}
```

论文里的例子是用户和 James 分手后, MemGPT 需要把 working context 里关于 James 的状态更新掉。否则 agent 后面继续问 “How is James?” 就会显得失忆或冒犯。

3.4 FIFO queue

FIFO queue 存放滚动消息历史, 包括:

- 用户消息。
- assistant 输出。
- system messages。
- memory warnings。
- function call inputs。
- function call outputs。

队列第一项还保存 recursive summary, 用来概括已经被挤出主上下文的旧消息。

它的作用不是永久记忆, 而是 “当前还在 prompt 里的一段连续交互轨迹”。

3.5 Recall storage

Recall storage 是消息数据库。Queue manager 会把用户输入和模型输出写入 recall storage。当旧消息从 FIFO queue 被 evict 后，它们不再直接可见，但仍然能被 recall function 查回来。

对话场景中，recall storage 解决的是：

- “我之前说过什么？”
- “上次我们聊到哪个地点？”
- “某个事件发生在哪次会话？”

3.6 Archival storage

Archival storage 是外部文档或任意长文本对象的存储。论文的文档 QA 实验里，Wikipedia 文档被加载到 archival storage，并通过 vector search 检索。

对比 recall storage：

- Recall storage 偏 conversation history。
- Archival storage 偏 uploaded documents, databases, arbitrary text objects。

两者都在 main context 外，但用途不同。

3.7 Function executor

LLM 的 completion tokens 会被 parser 解释成函数调用。Function executor 会：

- 验证函数名和参数。
- 执行 memory read, write, search, replace。
- 把函数结果或 runtime error 再放回 main context。

这个反馈很重要。模型不是一次性“想象自己调用了工具”，而是能看到工具返回值，然后继续调整行为。

3.8 Heartbeat and yield

很多任务需要连续调用多个函数。例如：

- 搜索第一页结果。
- 没找到答案。
- 请求第二页。
- 找到中间 key。
- 用中间 key 再查一次。

MemGPT 用 `request_heartbeat=true` 表示函数完成后立刻再触发一次 LLM inference，让 agent 可以继续链式调用函数。

如果没有 heartbeat，系统就是 yield：暂停处理，等下一个外部事件再触发。

这一点把 MemGPT 和普通 single-shot RAG 区分开来。普通 RAG 往往检索一次就回答；MemGPT 可以边查边想，直到它决定该交还控制权。

4. 系统结构重建

4.1 状态变量

你可以把 MemGPT 的运行时状态写成：

$S_t = (I, W_t, Q_t, R_t, A_t)$

I = read-only system instructions

W_t = working context at time t

Q_t = FIFO queue at time t

R_t = recall storage

A_t = archival storage

一次 LLM inference 真正看到的是:

$\text{prompt}_t = I + W_t + Q_t$

而 R_t 和 A_t 不会自动进入 prompt。它们必须通过函数调用移动一部分内容到 Q_t 或 W_t ，才会被 LLM 看见。

这就是 paper 里 “out-of-context data must be explicitly moved into main context” 的核心意思。

4.2 Context budget

设底层模型最大上下文长度为 C 。MemGPT 必须让:

$\text{tokens}(I) + \text{tokens}(W_t) + \text{tokens}(Q_t) \leq C$

但这个公式只是表面约束。真正工程上更难的是:

- I 里有很长的系统说明和函数 schema。
- W_t 不能无限增长，否则 persona 和用户事实会挤掉对话。
- Q_t 随 turn 增长，最容易爆。
- function outputs 也会消耗上下文。
- 搜索结果必须分页，否则一次 retrieval 就可能爆。

因此 MemGPT 需要 queue manager 监控 token pressure。

4.3 Queue manager 的控制规则

论文里给出一组非常具体的启发式设计:

- 当 prompt tokens 超过 warning token count, 例如 70 percent context window, queue manager 插入 memory pressure warning。
- warning 的作用是提醒 LLM 趁消息还在 prompt 里, 把重要内容写到 working context 或 archival storage。
- 当 prompt tokens 超过 flush token count, 例如 100 percent context window, queue manager flush queue。
- flush 时会 evict 一部分消息, 例如 50 percent context window。
- 被 evict 的消息会进入 recursive summary。
- 被 evict 的原始消息仍保存在 recall storage, 之后可以通过函数读回。

这个机制很像 OS 在内存快满时先给应用压力信号, 再真正回收页面。

4.4 事件驱动

MemGPT 不是只在用户发消息时运行。论文把 event 泛化成多种输入:

- user message。
- system message。

- memory pressure warning。
- 用户上传文档完成。
- 用户登录提醒。
- 定时事件。

事件会被 parser 转成纯文本消息，append 到 main context，然后触发 LLM inference。

这对 agent 很重要: 一个长期 agent 不能只被用户消息驱动，它可能需要在用户不说话时做维护、总结、检索、同步和自我更新。

4.5 Function call 控制流

一次 MemGPT 循环可以写成:

```
external event
-> event parser
-> append message to FIFO queue
-> build prompt from system, working context, FIFO
-> LLM processor generates completion
-> parser checks whether completion is function call
-> function executor runs memory operation
-> function result or error is appended to queue
-> if heartbeat: run LLM processor again
-> else: yield to user or wait for next event
```

从学习角度看，这就是 ReAct 的 thought-action-observation loop 在 memory management 场景中的系统化版本。

5. 张量和 token 级别怎么理解

这篇论文没有复杂训练 loss，也没有新 attention kernel，所以“张量级理解”要换一种读法: 关注 token budget 和可见性。

5.1 LLM 能看到什么

Transformer 每次推理只能对 prompt_t 里的 token 做 attention:

```
visible_tokens_t = tokenize(I + W_t + Q_t)
```

外部存储里的内容不在 attention matrix 里。

如果一条旧消息在 recall storage 里，但没有被 recall function 取回，它对当前 logits 没有直接影响。

因此 MemGPT 的关键动作不是“存了就有用”，而是:

```
stored outside context
-> selected by function call
-> inserted into main context
-> visible to attention
-> affects next completion
```

5.2 Attention 视角下的 main memory

普通 fixed-context chat:

```
attention input = recent messages only
early fact evicted
model cannot attend to early fact
```

MemGPT:

```
early fact evicted
fact stored in recall or archival storage
user asks related question
LLM calls search
search result inserted into queue
model attends to retrieved fact
answer uses old fact
```

所以 MemGPT 不是让 attention 矩阵变大, 而是让 “哪些 token 有资格进入 attention” 变成一个可管理的系统问题。

5.3 Working context 和 recall 的信息粒度

Working context 更像 compressed state:

```
human: Alice prefers Anthropic Claude.
persona: helpful technical tutor.
```

Recall storage 更像 raw event log:

```
turn 1 user: Hi, my name is Alice...
turn 2 assistant: ...
turn 3 user: ...
```

这两种粒度不能互相替代:

- 只有 working context, 容易丢细节和时间顺序。
- 只有 recall storage, 每次都要检索, 成本高且噪声大。
- 两者结合, 才像 OS 里的 register, RAM, disk 分层。

6. 方法细节逐段拆

6.1 Main context 的三段

论文把 prompt tokens 分成三段, 这是整篇方法的地基:

```
main context
  section 1: system instructions
  section 2: working context
  section 3: FIFO queue
```

新手要注意: working context 不是最近对话, FIFO queue 才是最近对话。Working context 更像一个可编辑的状态板。

6.2 External context 的两类

外部上下文可以是:

recall storage:
 conversation message database
 used for old chat turns

archival storage:
 arbitrary-length text objects
 used for documents and uploaded knowledge

LLM 无法直接 attend 到 external context, 必须发起函数调用。

6.3 Memory pressure warning

当 queue 变长, 系统会先告警。告警的价值在于它发生在 flush 前:

messages still visible
 -> warning says context is getting full
 -> LLM can decide what to save
 -> important facts move to working or archival memory
 -> later flush loses fewer important details

如果没有 warning, 系统只能机械 evict。那就像滑动窗口一样, 重要事实和废话一起丢。

6.4 Recursive summary

Flush 后, 旧消息不会完全消失。Queue 的第一项会维护 recursive summary。

这带来一个折中:

- summary 保留大意, 便宜。
- raw messages 保存在 recall storage, 可查。
- main context 不再被所有原文撑爆。

但 summary 也可能错。论文并没有证明 recursive summary 永远可靠; 它只是系统中的一个实用压缩层。

6.5 Pagination

检索结果也必须受 token budget 管理。MemGPT 的 memory retrieval 会分页, 避免一次 retrieval 把 context 塞爆。

文档 QA 例子里, agent 可以调用:

```
archival_storage.search("nobel physics")  
archival_storage.search("nobel physics", page=2)
```

这说明检索本身成为多步行为。模型不只是问数据库一次, 而是可以翻页、改 query、再翻页。

6.6 Function errors 也是反馈

Function executor 会把 runtime errors 也反馈给 processor, 例如:

- 参数格式错误。
- working context 已满。
- search 返回太多结果。
- search 没找到结果。

这让模型可以从工具反馈中修正行动。这个思想和后来 agent 工程里的 tool error recovery 很接近。

7. 和 ReAct, Toolformer, MCP 的关系

你刚读完 ReAct 和 Toolformer 后再读 MemGPT, 应该这样连接:

7.1 ReAct

ReAct 强调:

```
Thought
Action
Observation
next Thought
```

MemGPT 把 Action 的一个重要子集固定为 memory operations:

```
memory insert
memory search
working context replace
recall search
archival search
```

7.2 Toolformer

Toolformer 研究模型如何通过自监督数据学会 API calls。

MemGPT 默认使用已经具备 function calling 能力的 LLM, 然后用 system instructions 和 function schema 让它管理 memory。

也就是说:

- Toolformer 关心 “模型怎么学会调用工具”。
- MemGPT 关心 “工具调用能力可以组成怎样的 memory OS”。

7.3 MCP-style tool interface

现代 agent 工程会把工具和资源封装成统一协议。MemGPT 的 memory functions 可以看成是一个早期范式:

```
agent does not directly own all data
agent calls a structured interface
interface returns bounded result
result enters context
agent decides next call
```

这正是后续 MCP-style thinking 的前身: 工具接口、资源边界、分页、错误反馈、权限控制和可观察性都很重要。

8. 实验总览

论文评估两个大方向:

- Conversational agents。

- Document analysis。

它不是只展示一个 demo，而是做了四组关键实验：

- Deep Memory Retrieval, 简称 DMR。
- Conversation opener。
- Multi-document question answering。
- Nested key-value retrieval。

下面逐个拆。

9. 实验一：Deep Memory Retrieval

9.1 任务是什么

DMR 基于 Multi-Session Chat dataset。原数据包含多 session 对话，人类扮演固定 persona。

作者构造一个新的 session 6，让用户问一个必须依赖 session 1 到 session 5 旧对话才能回答的问题。

这个任务考的 not “模型会不会聊天”，而是：

Can the agent recall a narrow fact from prior conversations?

9.2 Baseline 怎么设

固定 context baseline 不能看到完整历史，只能看到过去五个 conversation 的 lossy summarization，用来模拟递归摘要。

MemGPT 则可以访问完整历史，但不能直接塞进 context，必须通过 paginated search 查询 recall memory。

这个对比很合理，因为它不是让 MemGPT 白拿完整 prompt，而是让它通过系统机制把旧信息调回来。

9.3 怎么评分

论文使用：

- ROUGE-L recall。
- LLM judge 判断 generated response 是否和 gold answer 一致。

作者特别指出，生成回答通常比 gold answer 更长，因此用 ROUGE-L recall 比直接精确匹配更合适。

9.4 结果数字

Table 2 的结果：

- GPT-3.5 Turbo baseline: accuracy 38.7 percent, ROUGE-L recall 0.394。
- GPT-3.5 Turbo with MemGPT: accuracy 66.9 percent, ROUGE-L recall 0.629。
- GPT-4 baseline: accuracy 32.1 percent, ROUGE-L recall 0.296。
- GPT-4 with MemGPT: accuracy 92.5 percent, ROUGE-L recall 0.814。
- GPT-4 Turbo baseline: accuracy 35.3 percent, ROUGE-L recall 0.359。
- GPT-4 Turbo with MemGPT: accuracy 93.4 percent, ROUGE-L recall 0.827。

9.5 这个结果证明了什么

它证明在需要回忆窄事实的长对话场景中，外部 memory 加主动 recall 明显优于固定 context 的摘要 baseline。

尤其 GPT-4 和 GPT-4 Turbo 加 MemGPT 后从约 30 percent 多 accuracy 到 90 percent 多 accuracy，说明这个任务的瓶颈不是纯语言能力，而是“能否把旧事实调回当前 context”。

9.6 这个结果没有证明什么

它没有证明：

- 所有长期记忆问题都解决了。
- LLM judge 没有偏差。
- 检索不会错。
- 写入 memory 永远正确。
- 用户隐私和安全边界自动成立。

DMR 的价值是清晰暴露 memory recall 能力，而不是覆盖所有真实 agent 风险。

10. 实验二: Conversation opener

10.1 任务是什么

Conversation opener 评估 agent 在新 session 开头能否主动生成更个性化、更有参与感的开场白。

一个好的 opener 应该引用过去积累的 persona 或用户信息，而不是泛泛说 “How can I help?”。

10.2 指标是什么

论文用 similarity score:

- SIM-1: 和 persona label 的相似度。
- SIM-3: 和多个 persona labels 的相似度。
- SIM-H: 和 human-created opener 的相似度。

10.3 结果数字

Table 3 报告:

- Human: SIM-1 0.800, SIM-3 0.800, SIM-H 1.000。
- MemGPT with GPT-3.5 Turbo: SIM-1 0.830, SIM-3 0.812, SIM-H 0.817。
- MemGPT with GPT-4: SIM-1 0.868, SIM-3 0.843, SIM-H 0.773。
- MemGPT with GPT-4 Turbo: SIM-1 0.857, SIM-3 0.828, SIM-H 0.767。

10.4 怎么读这个结果

MemGPT 生成的 opener 可以比 human opener 更贴近 persona labels，但不一定更像 human-created opener。

这提醒我们:

- 自动指标能说明它引用了更多 persona 信息。
- 但 “更像 persona label” 不等于 “真实用户更喜欢”。

- 论文也观察到 MemGPT opener 往往更 verbose, 会覆盖更多 persona 信息。

这组实验的贡献是证明 working context 中保存用户信息可以改善 personalization, 但它的证据强度不如 DMR 那么直接。

11. 实验三: Multi-document QA

11.1 任务设置

论文使用来自 Liu et al. 的 retriever-reader document QA 任务:

- 问题来自 NaturalQuestions-Open。
- 检索器从 Wikipedia 中选相关文档。
- LLM reader 根据文档回答。
- 评估 accuracy 随 retrieved documents 数量 K 的变化。

作者使用 late 2018 Wikipedia dump, 抽样 50 个问题。

检索使用 OpenAI text-embedding-ada-002 embedding 的 cosine similarity。MemGPT 的 archival memory 使用 PostgreSQL 加 pgvector, 并用 HNSW index 支持近似 sub-second query。

11.2 Baseline 和 MemGPT 差别

Fixed-context baseline:

```
retriever gets top K
documents are concatenated or truncated into prompt
LLM answers once
```

MemGPT:

```
all document embeddings loaded into archival storage
LLM calls archival_storage.search
search results enter main context
LLM can page through more results
LLM answers when evidence is enough
```

11.3 结果怎么解读

Figure 5 的核心结论:

- Fixed-context baseline 的上限被 context window 限制。
- K 小时, gold article 可能没有进 prompt。
- K 大时, 文档被截断, gold snippet 可能被截掉。
- MemGPT 能通过多次 archival search 扩展 effective context。
- GPT-3.5 版本的 MemGPT 明显退化, 论文归因于较弱的 function calling 能力。
- GPT-4 版本表现最好。

这里没有必要背 Figure 5 的曲线坐标。你要记住证据逻辑:

```
same retriever
fixed model sees only top K in prompt
MemGPT can query archive repeatedly
therefore MemGPT is less tied to one-shot context packing
```

11.4 重要限制

论文也承认 embedding-based similarity search 本身有限。Gold document 往往不在前十几个结果里。

这说明 MemGPT 不是神奇地解决检索。它只是允许 agent 多次检索和分页。如果 retriever ranking 极差，agent 仍然可能找不到答案，或者过早停止翻页。

12. 实验四: Nested key-value retrieval

12.1 为什么要做这个任务

普通 KV retrieval 只要求:

```
given key k1
return value v1
```

Nested KV 要求:

```
k1 -> k2 -> k3 -> final value
```

这逼迫 agent 连续查询，而不是一次读完所有内容。

12.2 实验设置

论文构造:

- 140 个 UUID key-value pairs。
- 约 8k tokens，匹配 GPT-4 baseline context。
- nesting levels 从 0 到 4。
- 每个设置采样 30 种 ordering configurations。

12.3 结果怎么读

Figure 7 的核心观察:

- GPT-3.5 和 GPT-4 在原始非嵌套 KV 上表现不错，但嵌套任务会迅速掉。
- GPT-3.5 在 1 nesting level 就到 0 percent accuracy，主要失败模式是返回第一跳 value。
- GPT-4 和 GPT-4 Turbo 更强，但到 3 nesting levels 也掉到 0 percent。
- MemGPT with GPT-4 基本不受 nesting levels 影响。
- MemGPT with GPT-4 Turbo 和 GPT-3.5 比各自 baseline 更好，但到 2 nesting levels 会开始掉，因为它们没有做足够多 lookup。

这个实验很漂亮，因为它直接测试 function chaining。若模型只会一次性看 prompt，它会在多跳任务里混乱；若 agent 能链式 search，就能一步步走到最终值。

13. 论文贡献重新归纳

13.1 贡献一: Virtual context management

MemGPT 提出一个系统层方法，让固定 context LLM 通过 memory hierarchy 和 function calls 管理超出 context 的信息。

13.2 贡献二: OS-inspired memory hierarchy

它把 memory 分成:

- Main context.
- Working context.
- FIFO queue.
- Recall storage.
- Archival storage.

这套分层比“向量库加 RAG”更细,因为它明确区分当前可见状态、消息历史、外部文档和控制事件。

13.3 贡献三: Event-based control flow

Memory pressure, user message, system alert, document upload, timed event 都可以触发 agent 处理。

这让 MemGPT 更像 agent runtime, 而不是简单的 prompt 模板。

13.4 贡献四: Function chaining

通过 heartbeat, MemGPT 可以在交还用户前连续调用多个 memory functions。

这个能力在 nested KV 和 document QA 中非常关键。

13.5 贡献五: 两类实验验证

论文既验证对话记忆,也验证文档分析。DMR 的数字尤其强, Nested KV 则清楚展示了多跳函数调用的必要性。

14. 设计动机和技术权衡

14.1 为什么让 LLM 自己管 memory

规则系统可以写:

```
if message contains name: save name
if message contains preference: save preference
if context too long: summarize
```

但真实对话的“值得记”很难只靠规则。MemGPT 让 LLM 根据语境决定:

- 这是不是重要事实。
- 应该写到 working context 还是 archival storage。
- 是否需要 search recall。
- 是否需要继续翻页。
- 是否该更新旧状态。

好处是灵活,坏处是依赖模型的工具调用质量。

14.2 为什么不是单纯 summary

Summary 适合保留大意,不适合保留所有可被点名追问的窄事实。

DMR 就是用来打这个痛点: 用户可能问非常具体的旧细节。摘要如果没写进去,就找不回来了。

MemGPT 用:

- recursive summary 保留粗略轨迹。
- recall storage 保留原始消息。
- search function 按需恢复细节。

14.3 为什么不是单纯 vector DB

向量库只是 external storage 的一种实现。MemGPT 还需要:

- 主上下文结构。
- 写入和更新策略。
- token pressure 控制。
- function parser。
- error feedback。
- pagination。
- heartbeat chaining。

没有这些, 向量库只是 “能查东西”, 不是 “会管理记忆”。

14.4 为什么不是直接无限 context

即使模型 context 很长, 也有:

- 成本问题。
- 延迟问题。
- 中间信息利用问题。
- 多文档无限增长问题。
- 跨 session 持久化问题。
- 用户隐私和权限问题。

MemGPT 的主张是: context window 继续变大也有用, 但 memory hierarchy 仍然必要。

15. 本仓库代码对应关系

15.1 letta_mock.py

对应论文的 main context 和 archival memory 最小版:

- core 对应 working context。
- recent 对应 FIFO queue 的简化版。
- archive 对应 archival storage。
- add_message 超过 max_recent 后把旧消息转入 archive。
- archival_search 用 hash embedding 做 toy retrieval。
- build_main_context 把 system, core, recent 拼成当前 prompt。

读这个文件时, 重点看:

```
mem.add_message("user", "msg 1")
mem.core_replace("human", "User name is Alice")
found = mem.archival_search("Alice engineer", k=3)
ctx = mem.build_main_context()
```

它不是完整 MemGPT, 但能让你看到 main/external context 的最小分界。

15.2 memgpt_virtual_context.py

这是为这篇 guide 补的 toy implementation, 专门模拟论文里最容易空谈的系统机制:

- VirtualContext.prompt_tokens 估计当前 prompt budget。
- add_event 把事件写入 FIFO 和 recall storage。
- _check_pressure 在超过 warning ratio 时返回 warning, 在超过 capacity 时 flush。
- recursive_summary 保存被 evict 消息的摘要。
- recall_search 从 recall storage 查旧消息。
- nested_kv_lookup 模拟 heartbeat function chaining 的多跳 KV lookup。
- paginated_search 模拟 document QA 的分页检索。

最小实验:

```
from memgpt_virtual_context import VirtualContext, nested_kv_lookup

vc = VirtualContext(capacity_tokens=45, system_tokens=5)
vc.core_replace("human: Alice prefers Anthropic Claude")

for i in range(12):
    report = vc.add_event("user", f"turn {i}: Alice discussed RAG")

print(report.warned, report.flushed)
print(vc.recursive_summary)
print(vc.recall_search("RAG"))

path, final = nested_kv_lookup({"k1": "k2", "k2": "answer"}, "k1")
print(path, final)
```

如果你只跑一个文件, 就跑这个。它直接对应论文的 memory pressure, flush, recall, chaining。

15.3 context_mgmt.py

这个文件对应长对话管理的常见替代方案:

- sliding window。
- importance pruning。
- rolling summary。
- RAG-over-history。

它帮助你理解 MemGPT 相对这些方法的差别:

```
sliding_window: keep latest only
rolling_summary: compress old turns
rag_history: retrieve relevant old turns
MemGPT: combine context structure, storage, functions, pressure warnings, and chaining
```

15.4 capstone_memory_chat.py

这个 capstone 把长期用户偏好保存到 semantic profile:

- Turn 1 用户说自己叫 Alice, 偏好 Anthropic Claude。
- Turn 2 到 Turn 9 问很多无关问题。
- Turn 10 问 preferred LLM。

- 系统应该从 semantic profile 回答 Anthropic Claude。

它对应 DMR 的简化精神: 后面的问题需要回忆前面 turn 的事实。

15.5 vector_store.py, episodic_memory.py, semantic_memory.py

这些文件补足真实 memory agent 常见存储层:

- Vector store 做相似度检索。
- Episodic memory 保存带时间戳的事件。
- Semantic memory 保存事实和 KG triples。

MemGPT 论文没有把这些都工程化成生产系统, 但你的学习仓库需要这些模块, 才能从论文走到可用 agent。

16. 本地运行路径

先验证环境:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python learning\agent-memory-context\environment\verify_env.py
```

再跑专题测试:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python learning\agent-memory-context\src\tests\test_memory.py
```

再单独跑 MemGPT toy:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python learning\agent-memory-context\src\memgpt_virtual_context.py
```

再跑 capstone:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python -c "import sys; sys.path.insert(0,'learning/agent-memory-context/src'); from capstone_memory_chat import main; main()"
```

读代码顺序:

1. letta_mock.py
2. memgpt_virtual_context.py
3. context_mgmt.py
4. episodic_memory.py
5. semantic_memory.py
6. capstone_memory_chat.py

17. 新手最容易误读的点

17.1 “MemGPT 等于 RAG”

不对。RAG 通常是检索增强生成, 很多实现是一轮检索加一轮回答。

MemGPT 是 memory OS:

- 有 main context 分区。
- 有 external context 分层。
- 有 queue manager。

- 有 memory pressure。
- 有 function executor。
- 有 heartbeat chaining。
- 有 recall 和 archival 两类存储。

RAG 是其中一个可用组件。

17.2 “MemGPT 拥有无限上下文”

也不对。MemGPT 提供的是 virtual context illusion。

底层 LLM 每次仍然只能看固定 context。它只是通过外部存储和函数调用，让系统可以在更大的信息空间里按需调页。

17.3 “只要有 summary 就够”

DMR 说明 summary 不一定够。用户可能问一个非常窄的旧事实，而摘要没有保留。

17.4 “工具调用模型越强，MemGPT 就一定越好”

工具调用能力很关键，但不唯一。检索质量、memory 写入质量、分页策略、提示设计、错误恢复和隐私边界都会影响结果。

17.5 “写入 memory 永远是好事”

错误写入会污染后续行为。例如把临时情绪当长期偏好，把过期关系状态当当前事实，或者跨用户串记忆。

真实系统必须有：

- update logic。
- delete logic。
- user controls。
- privacy boundary。
- conflict resolution。

18. 局限性

18.1 依赖 function calling 能力

论文结果已经显示 GPT-3.5 版本的 MemGPT 在文档 QA 和 nested KV 上明显弱于 GPT-4 版本。

如果底层模型不会稳定调用工具、不会继续翻页、不会根据 error 修正参数，MemGPT 的系统设计就发挥不出来。

18.2 检索仍然是瓶颈

文档 QA 中，gold document 经常不在前十几个检索结果里。MemGPT 可以翻页，但不保证会翻到足够远。

也就是说，MemGPT 缓解 context packing 问题，但不自动解决 retriever ranking。

18.3 评测依赖 LLM judge

DMR 和 document QA 都使用 LLM judge。LLM judge 在很多任务上 useful，但会有 bias、verbosity sensitivity 和一致性问题。

读实验时不要把 judge accuracy 当成绝对真理。

18.4 Memory 写入和更新没有形式化保证

论文的 memory edits 是 LLM 自主生成的。它没有给出一个可证明正确的写入策略。

真实产品中，错误 memory 可能比没有 memory 更糟。

18.5 安全和隐私只是开始

长期记忆 agent 必须处理：

- 用户是否同意保存。
- 哪些内容不能保存。
- 如何查看和删除 memory。
- 多用户 memory 隔离。
- prompt injection 是否能诱导 agent 泄露 memory。
- 敏感信息是否应该进入 retrieval index。

MemGPT 论文主要证明系统可行性，没有把这些都解决。

19. 对现在的意义

19.1 为什么 2026 仍然要学

即使今天上下文更长，agent 仍然需要 memory hierarchy。

原因很朴素：

- 上下文再长也有成本。
- 多 session 记忆不能每次重传。
- 用户 profile 需要持久化。
- 文档库和工具结果需要分页。
- 长任务需要状态管理。
- 安全边界需要显式接口。

所以 MemGPT 的 OS 类比仍然有学习价值。

19.2 它给 agent 工程的启发

一个可用 agent 不能只会回答，还要有运行时：

- event loop。
- state。
- tool schema。
- memory tiers。
- retrieval policy。
- summarization policy。
- error recovery。

- observability。
- permission model。

MemGPT 是把这些问题放到一张系统图里的早期代表。

19.3 它和本仓库学习路线的关系

在你的 LLM 全栈学习仓库里，这篇论文连接多个专题：

- agent-foundations: ReAct 的行动循环。
- tool-use-mcp: Toolformer 和工具接口。
- agent-memory-context: 长期记忆和上下文管理。
- rag-essential: archival search 的文档检索。
- inference-engine-core: KV cache 和真正的底层 memory management。
- safety-defense: memory privacy 和 prompt injection。

读 MemGPT 的时候，你要把它当成 agent runtime 论文，而不是单点算法论文。

20. 建议你怎么用 AI agent 学这篇

20.1 不要让 agent 直接总结

不要问：

总结一下 MemGPT。

这会得到很漂亮但很浅的答案。

改成：

请围绕 MemGPT 的五个机制追问我：

1. main context
2. recall storage
3. archival storage
4. queue manager
5. function executor

一次只问一个问题。

我回答后，你必须指出我漏掉的是系统结构、控制流、实验数字还是局限性。

20.2 让 agent 考你状态转移

好问题：

假设 capacity 是 100 tokens，warning 是 70 tokens。

现在 FIFO queue 让 prompt 达到 75 tokens。

MemGPT 应该发生什么？

如果继续到 100 tokens，又应该发生什么？

被 evict 的消息去了哪里？

你必须答出：

- 先 memory pressure warning。
- LLM 有机会保存重要信息。
- 到 flush threshold 后 queue manager evict。
- 生成 recursive summary。

- 原始消息仍在 recall storage。

20.3 让 agent 考你实验

好问题:

DMR 为什么比普通聊天 benchmark 更能测试 memory?

GPT-4 baseline 为什么比 GPT-4 with MemGPT 差这么多?

这个实验没有证明什么?

你必须答出:

- DMR 问的是旧 session 的窄事实。
- baseline 只有 lossy summary。
- MemGPT 可以 paginated recall。
- 但它没有证明所有 memory 写入可靠, 也依赖 LLM judge。

20.4 让 agent 逼你连代码

好问题:

请把 MemGPT 的 warning, flush, recall, heartbeat 分别对应到本仓库哪个文件或函数。

如果没有对应实现, 请指出缺口。

你应该能说:

- memgpt_virtual_context.py 对应 warning, flush, recursive summary, recall, nested KV chaining。
- letta_mock.py 对应 core, recent, archive, build_main_context。
- context_mgmt.py 对应 sliding, summary, prune, rag_history。
- capstone_memory_chat.py 对应长期偏好 recall toy test。

21. 30 分钟学习任务

21.1 第 1 遍: 只画系统图

目标: 不看数字, 只画:

```
event
-> FIFO queue
-> main context
-> LLM processor
-> function executor
-> recall storage or archival storage
-> result back to FIFO queue
```

然后解释 working context 和 FIFO queue 的差别。

21.2 第 2 遍: 跑 toy code

运行:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python learning\agent-memory-context\src\memgpt_virtual_context.py
```


观察:

- 什么时候 warned。
- 什么时候 flushed。
- recursive summary 何时出现。
- recall_storage 是否还保留所有事件。
- nested KV path 是否一步步走到 final。

21.3 第 3 遍: 背实验数字

至少背 DMR 这组:

- GPT-3.5 baseline 38.7 percent, with MemGPT 66.9 percent。
- GPT-4 baseline 32.1 percent, with MemGPT 92.5 percent。
- GPT-4 Turbo baseline 35.3 percent, with MemGPT 93.4 percent。

然后讲清楚为什么 baseline 不是 “笨 baseline”，而是 lossy summary baseline。

21.4 第 4 遍: 改一个变量

打开 memgpt_virtual_context.py, 把:

```
warning_ratio = 0.70
```

改成 0.50 或 0.90, 观察 flush 前 warning 的触发时机。

学习目标不是 “得到更好结果”，而是理解 memory pressure warning 为什么要早于 flush。

21.5 第 5 遍: 闭卷复述

用 200 字回答:

MemGPT 为什么不是普通 RAG?

它如何用 OS 虚拟内存类比解决固定 context 限制?

它的 DMR 和 Nested KV 实验证明了什么?

如果你能答清楚, 才算真的进脑袋。

22. 一页纸复习版

22.1 最小机制

finite LLM context

-> split into system, working context, FIFO queue

external context

-> recall storage for messages

-> archival storage for documents

LLM output

-> parsed as function call

-> executor runs memory operation

-> result goes back into queue

memory pressure

-> warning at about 70 percent

-> flush at about 100 percent

-> evict about half and update recursive summary
function chaining
-> heartbeat triggers follow-up inference

22.2 最小证据

- DMR: MemGPT variants strongly outperform fixed-context summary baselines on old conversation recall.
- Conversation opener: MemGPT variants produce openers similar to persona labels and sometimes stronger than human opener on SIM-1 and SIM-3.
- Document QA: MemGPT can query archival storage repeatedly instead of packing fixed top-K documents into prompt.
- Nested KV: MemGPT with GPT-4 can keep doing multi-hop lookup where fixed-context baselines collapse.

22.3 最小警惕

- It is virtual context, not infinite attention.
- It depends on tool calling.
- Retrieval quality remains a bottleneck.
- Memory writes can be wrong.
- Privacy and deletion controls are production requirements.

23. 闭卷自测

1. Main context 的三段是什么?
2. Working context 和 FIFO queue 的差别是什么?
3. Recall storage 和 archival storage 的差别是什么?
4. Memory pressure warning 为什么要早于 flush?
5. 被 flush 的消息去了哪里?
6. request_heartbeat=true 解决什么问题?
7. DMR 任务为什么比普通聊天更适合测试长期记忆?
8. Table 2 里 GPT-4 with MemGPT 的 accuracy 是多少?
9. Document QA 里 fixed-context baseline 的上限为什么会被 context window 卡住?
10. Nested KV 为什么能测试 function chaining?
11. MemGPT 为什么不是普通 RAG?
12. 本仓库哪个文件最直接模拟 warning 和 flush?

24. 你应该带走的直觉

MemGPT 的核心直觉是:

LLM context window is not the whole memory system.
It is only the currently visible memory tier.

真正的 agent 需要管理:

- 哪些信息现在可见。
- 哪些信息暂时不可见但可检索。
- 哪些信息应该被压缩。
- 哪些信息应该被长期保存。

- 什么时候调用工具把信息调回。
- 什么时候停止检索并回答。

这就是 MemGPT 对 LLM agent 学习路线的意义: 它让你从 “写 prompt” 进入 “设计运行时”。